Mobile and Distributed Computing Systems

D7024E · Lecture 1 · Course Introduction

Johan Kristiansson · Carl Borngrund · Christoffer Fink



Let's start with a question

How many computers are involved
when you press play on **one** Netflix episode?

Let's start with a question

How many computers are involved
when you press play on **one** Netflix episode?

One big server somewhere in California?

Let's start with a question

How many computers are involved
when you press play on **one** Netflix episode?

One big server somewhere in California?

For your single click: **dozens**

DNS · API gateway · auth & recommendation services · a CDN cache **here in Sweden**

Let's start with a question

How many computers are involved
when you press play on **one** Netflix episode?

One big server somewhere in California?

For your single click: **dozens**

DNS · API gateway · auth & recommendation services · a CDN cache **here in Sweden**

For all ~300 million subscribers: **hundreds of thousands of servers.**

And when one of them dies mid-episode, you never even notice.

That trick — **many machines pretending to be one, reliable, machine** — is this course.

“But AI writes my code now…” — exactly. That’s the point.

- Claude Code or Copilot can generate a microservice in 30 seconds
  Typing was never the hard part
- What AI cannot decide for you: **the trade-offs.** Strong or eventual consistency? Retry or fail fast? One region or three? — each answer is cheap to type and expensive to get wrong
- AI amplifies intent. Without distributed-systems judgement, you ship confident-looking systems that fall over at the first network partition — **just faster than ever before**
- And AI is a distributed system: trained on tens of thousands of GPUs, served through load balancers, caches and consensus stores

That is why this may be the most important course of your degree: it trains the judgement layer above code generation — knowing what to build, what it costs, and how it fails.

Welcome to D7024E!

Your teachers

Johan Kristiansson — examiner

johan.kristiansson@ltu.se · office A2306

Carl Borngrund — teacher / TA

carl.borngrund@ltu.se · office A2307

Christoffer Fink — teacher / TA

christoffer.fink@ltu.se · office A2309

How this course runs

Lectures: on campus + live on Zoom,

One big **lab** (groups of 3)

Student **seminars** on research papers

Take-home **exam**

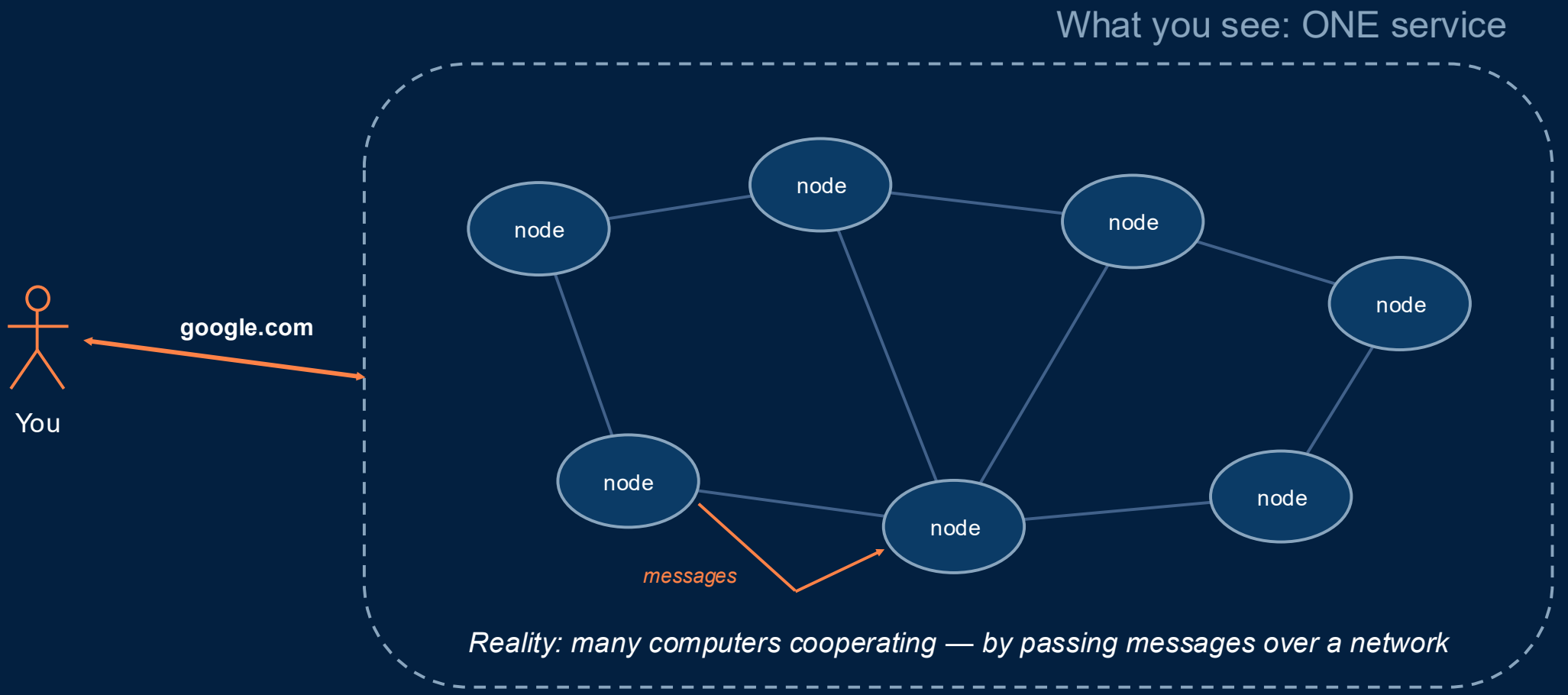
We will each introduce ourselves the first time we meet you.
Note: Christoffer will be remote only.

Part 1

What is a distributed system?

… and why they are everywhere, and why they hurt

What is a distributed system?



Independent computers · **no shared memory** · **no common clock** · only messages
yet to the user it appears as **one single coherent system.**

Examples of distributed systems

Google Docs
everyone edits, nobody waits
Consistency · CRDTs

Netflix
the video is already near you
Replication · caching · CDNs

BitTorrent / IPFS
files with no server at all
DHTs · Kademlia + THE LAB

Kubernetes / etcd
who is in charge when nodes die?
Leader election · Raft

Bitcoin / Ethereum
money with no bank
Byzantine consensus · PoW/PoS

WhatsApp
delivered — in order — exactly once?
RPC · pub-sub · gossip

Fortnite
100 players, 30 milliseconds
State sync · lag compensation

Google Search
the whole web in 200 ms
Scalability · trade-offs

Every card is a hard problem someone solved — and **every card is a lecture in this course.**

Why make a system distributed?

- **It already is.** Your phone talking to your friend's phone is a distributed system
- **Reliability.** One machine will crash. A thousand machines crash all the time — and the service survives. *(Netflix kills its own servers on purpose: Chaos Monkey.)*
- **Performance.** Light is slow: Luleå ↔ Sydney is ~300 ms round-trip.
  The fix: put the data **near the user.**
- **Scale.** The whole web does not fit on one disk; a World Cup final does not fit on one server.

Scaling: the cloud answer

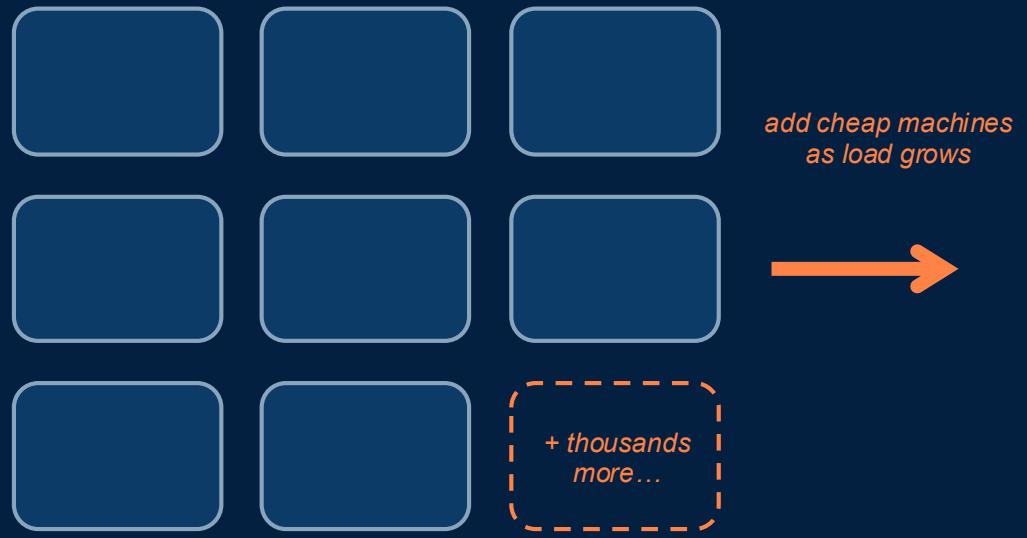
Scale UP

hardware limit (and your budget)



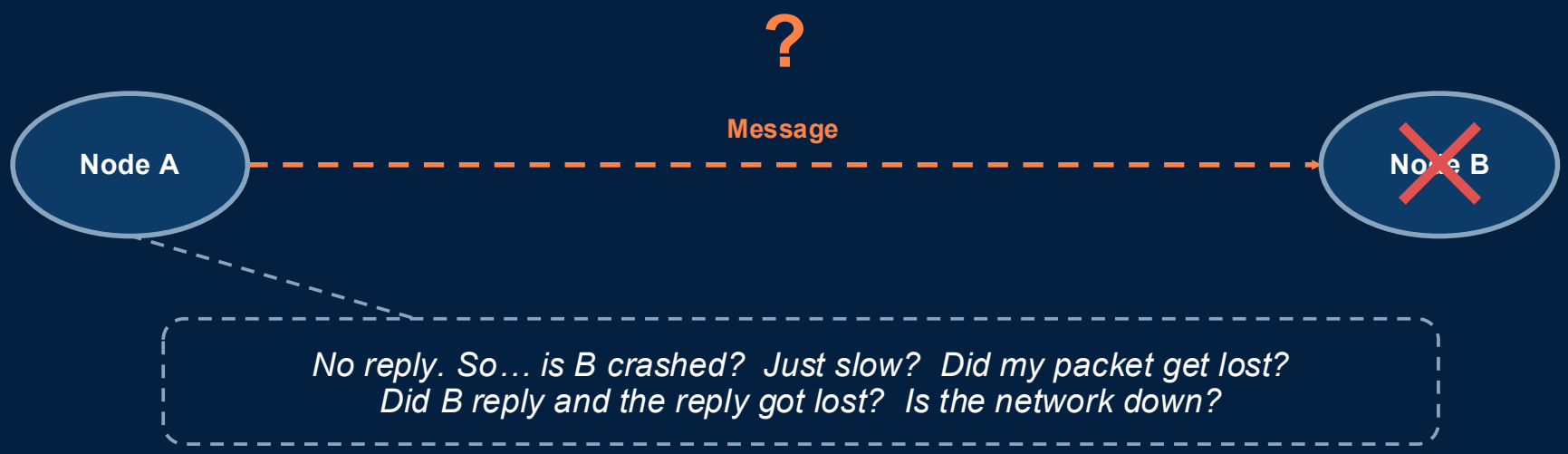
buy a faster machine…
until there is no faster machine

Scale OUT



how Google, Amazon — and your lab — scale
the catch: now it must be a distributed system

Many *(all?)* systems are already distributed



You can NEVER tell the difference → design for it: timeouts, retries, replication, k-redundancy

"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."
— Leslie Lamport

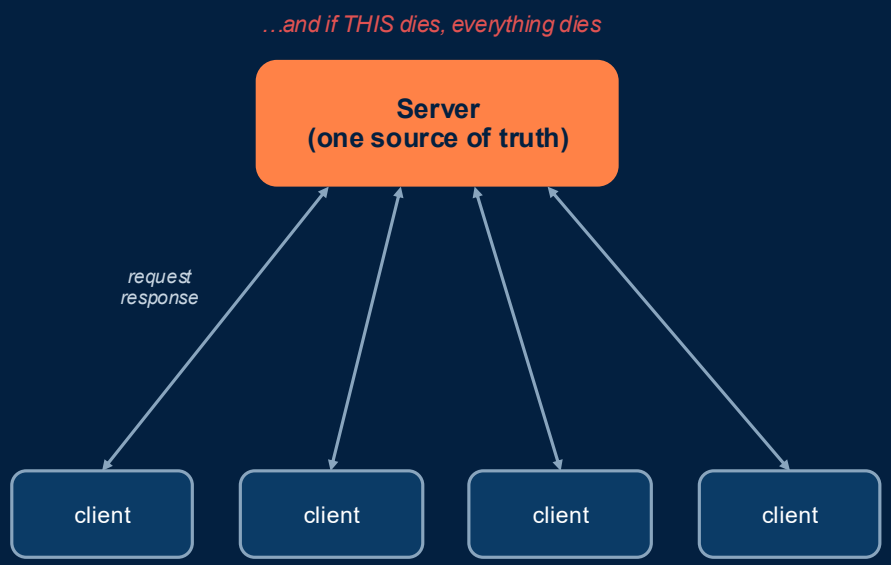
Part 2

Architectures

different ways to organise machines — each one a different trade-off

Every architecture is a different answer to the same two questions: ***where does state live, and who talks to whom?***

Architecture 1: Client–Server



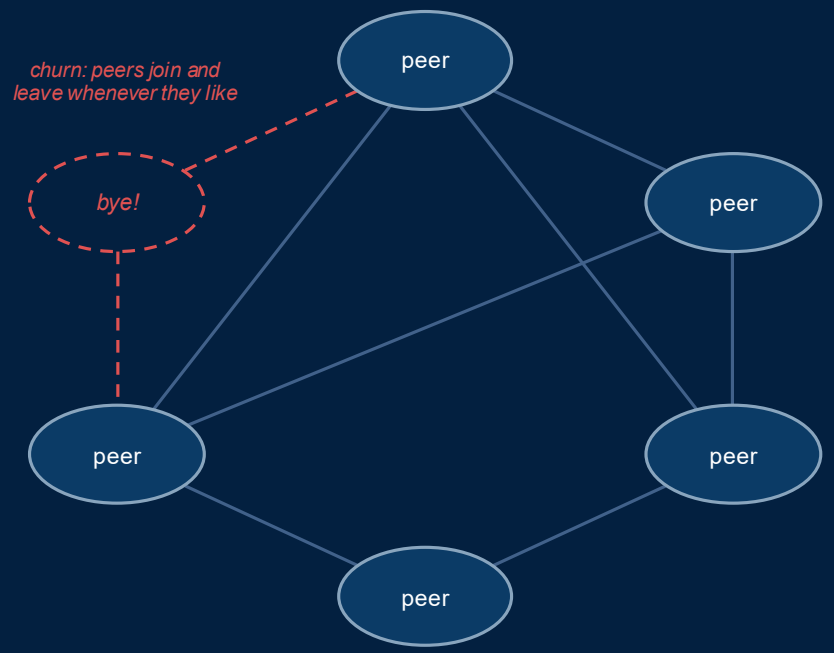
clients are thin; the server holds state, logic and the pager duty

+ Simple to build, debug and secure — one place to look
+ One source of truth: no consistency headaches
– Single point of failure and a scaling ceiling
– The server pays for everything — and is the target of every attack

Trade-off: simplicity vs. resilience & scale

In the wild: every classic website and app backend. The request/reply machinery (REST, RPC)

Architecture 2: Peer-to-Peer (P2P)



every node is client AND server — there is no boss to call

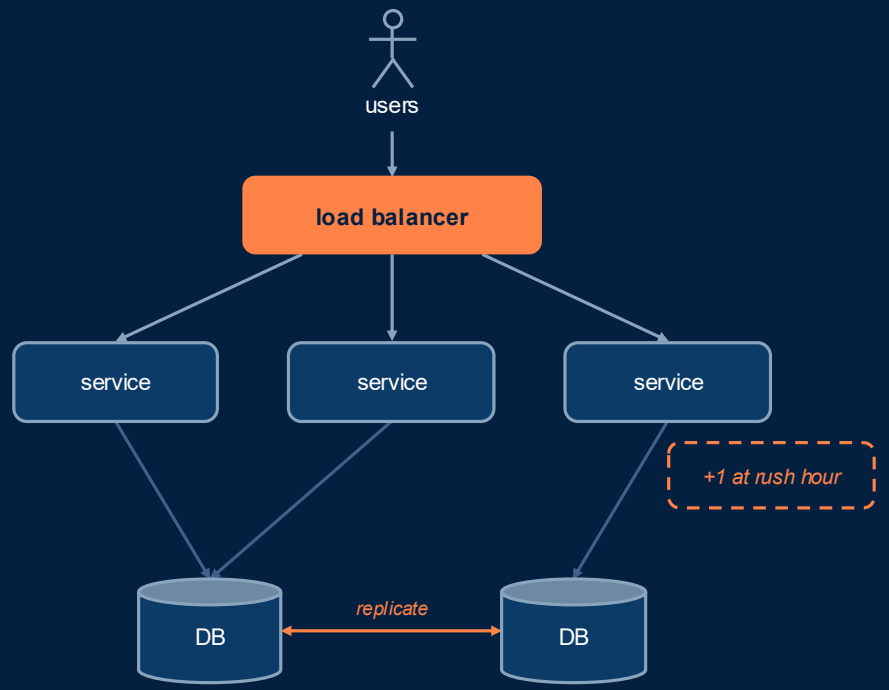
+ No single point of failure — and no hosting bill
+ Capacity grows with every user who joins
– Churn: peers vanish mid-conversation, constantly
– Discovery, NAT traversal, trust: who do you even ask?

Trade-off: resilience & cost vs. coordination complexity

BitTorrent, IPFS, blockchain networks

and your lab (Lecture 3).

Architecture 3: Cloud-native microservices



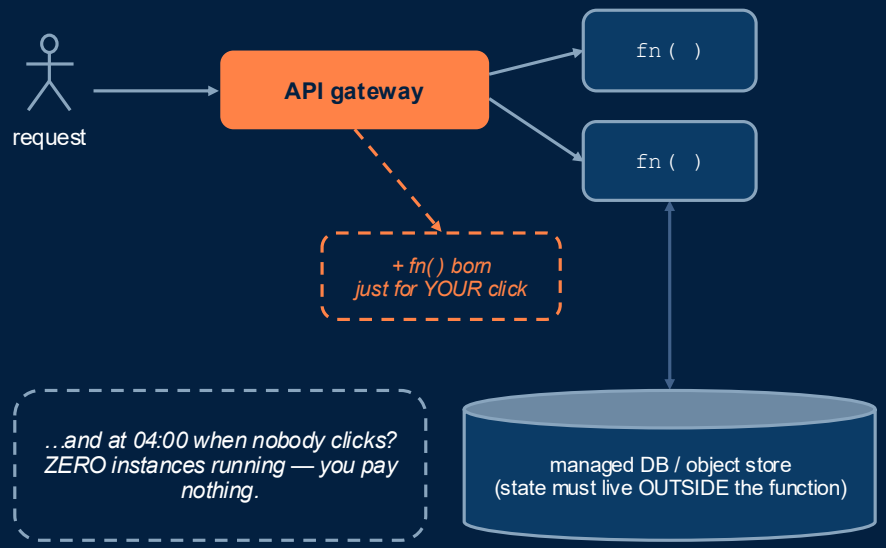
stateless services scale out; state lives in replicated, managed storage

+ Elastic: scale out at rush hour, scale in at 04:00
+ Managed: replication, balancing, monitoring as a service
– Vendor lock-in; the bill can explode at scale
– You rent your infrastructure — and your data lives in it

Trade-off: velocity & elasticity vs. control & cost

Netflix on AWS, Spotify on Google Cloud.

Architecture 4: Serverless (FaaS)



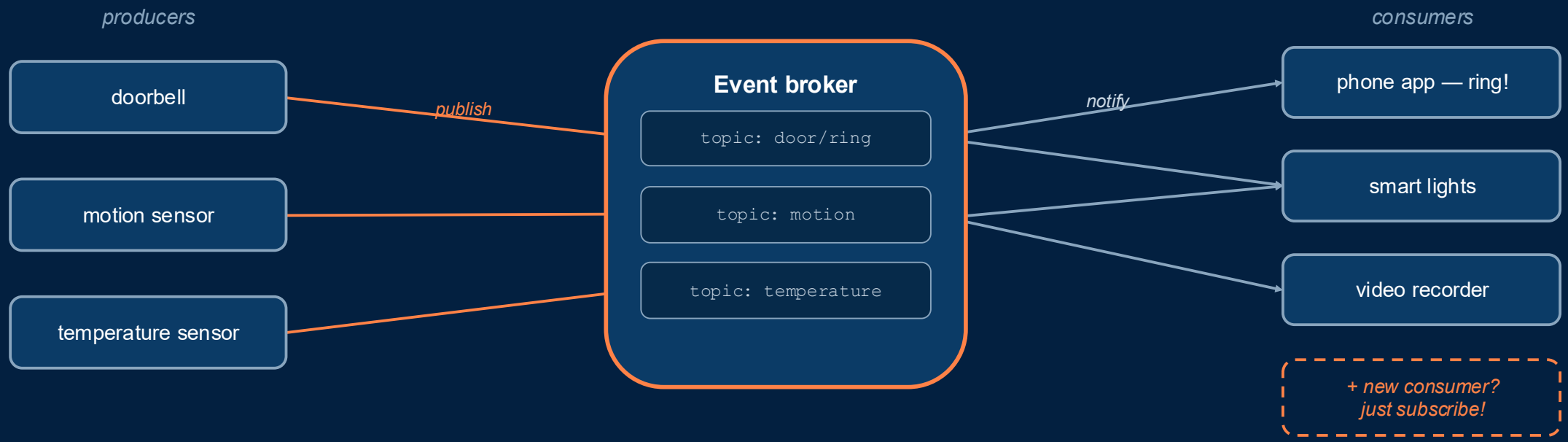
you write ONE function — the platform handles servers, scaling, restarts, the 3 a.m. pager

+ Zero ops — nobody patches servers at 3 a.m.
+ Scale to zero: a hobby project costs ~0 kr
+ Instant scale-out when you go viral
– Cold starts add latency; state must live elsewhere
– The deepest vendor lock-in of them all

Trade-off: operational simplicity vs. control & predictability

In the wild: AWS Lambda, Cloud Functions. And ColonyOS — coming up — is FaaS across cloud, edge and HPC.

Architecture 5: Event-driven



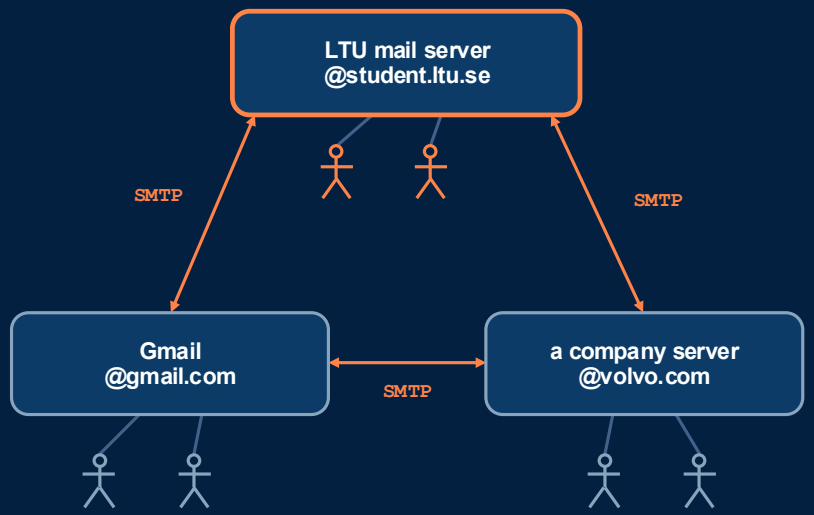
Nobody calls anybody: the doorbell has no idea your phone exists. Total decoupling — in space AND time.
MQTT, Kafka, pub-sub, gossip

+ total decoupling, easy to extend, elastic **–** debugging becomes archaeology; ordering & duplicates haunt you

Trade-off: flexibility vs. traceability

Smart home, Discord pings, Uber — one "ride requested" event fans out to pricing, matching, maps, billing.

Architecture 6: Federations



many independent servers, one open protocol, one global system — no "email headquarters" exists, yet any address reaches any inbox on Earth

+ No central owner — survives any single operator's death
+ Local autonomy + global reach, via open protocols
– Evolves at the speed of committee
– Spam & trust: everyone's problem, no one's job

Trade-off: autonomy & openness vs. uniformity & pace

*50 years old and still undefeated; Mastodon, Matrix. And **eduroam**: the wifi you are on right now is a federation.*

Part 3

Applications in the wild

the cloud, containers, Kubernetes, big data — where you will meet all of this professionally

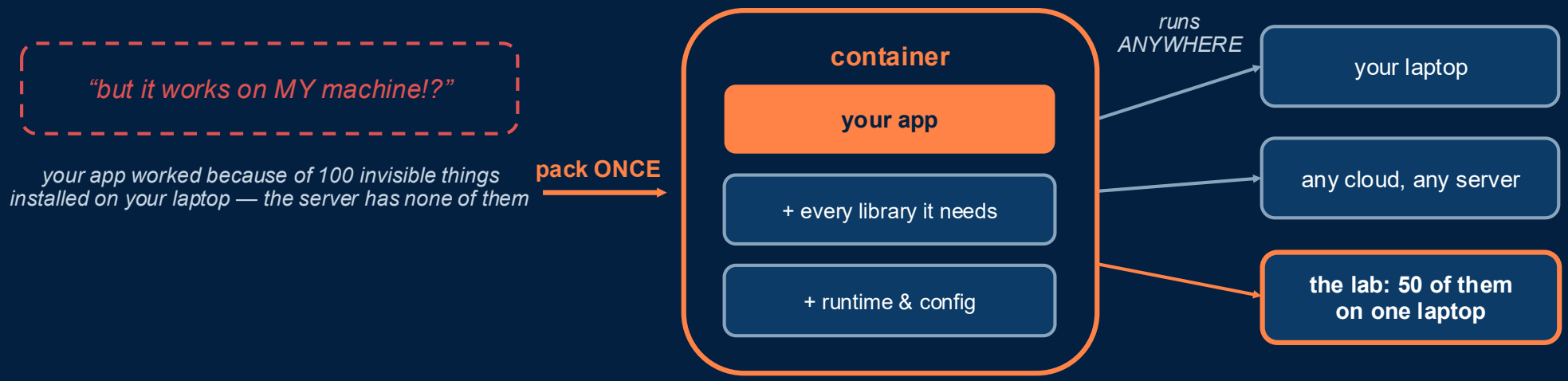
Use case: Is the cloud a distributed system?

***Yes* — the cloud is distributed systems sold by the hour.**

- Inside every AWS/Azure/GCP region: load balancers, replicated storage, consensus services — everything in this course, running at planetary scale.
- Why client–cloud won the 2000s: firewalls/NAT make direct peer connections painful, and businesses like to keep state (and customers) close.
- Why P2P is back anyway: BitTorrent, IPFS, blockchains, edge computing — **no owner, no single point of failure, no monthly bill.**

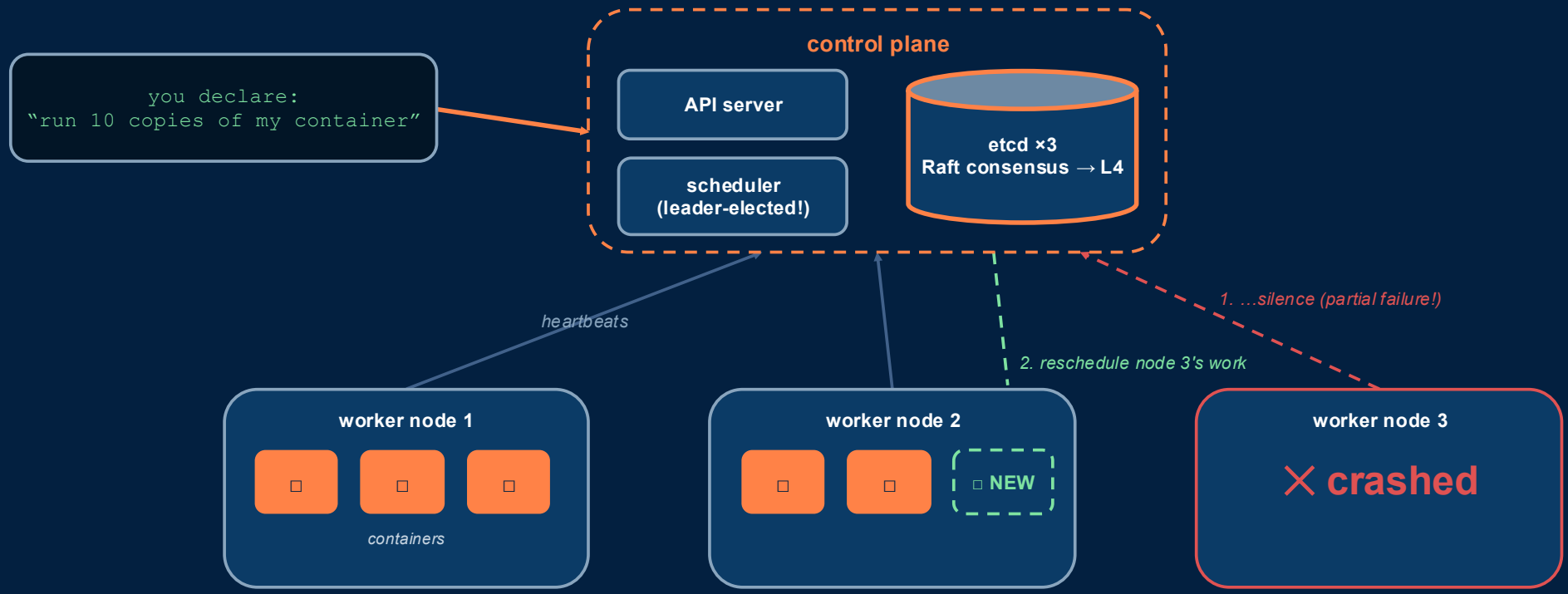
In the lab you build the radical version: a storage system with no server at all

Containers (e.g. Docker)



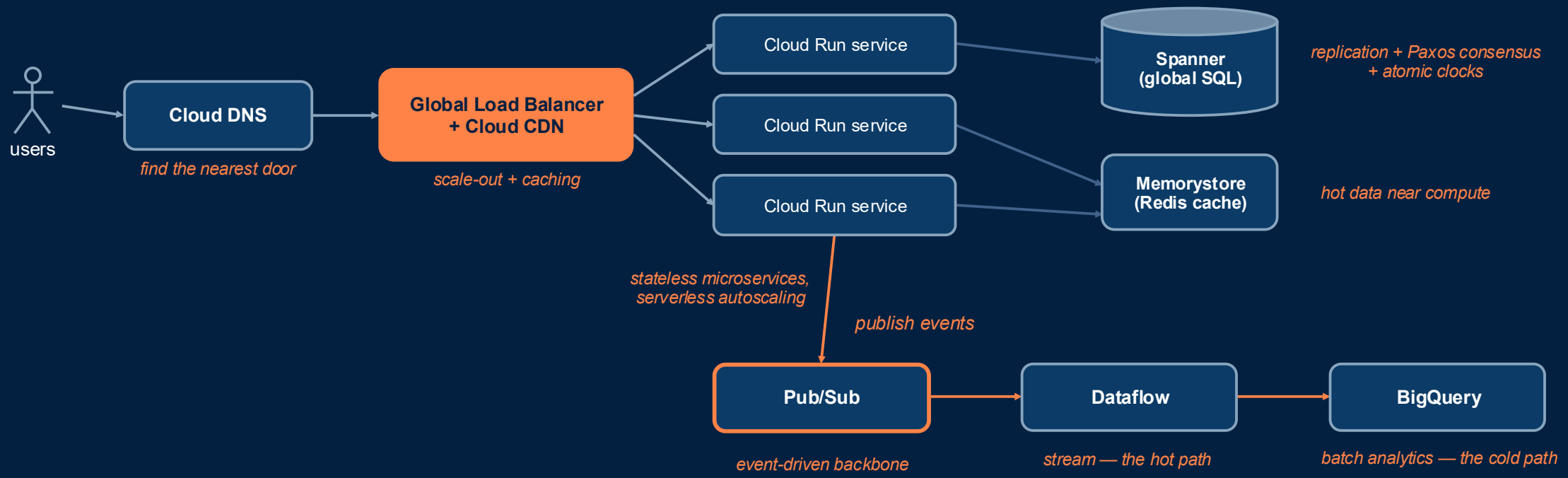
like a virtual machine, but it shares the host's kernel: starts in milliseconds, weighs megabytes — so one laptop can run 50 "computers".

Kubernetes: A cloud platform that is itself a distributed system



You say WHAT (desired state) — Kubernetes fights reality until it matches: failure detection by heartbeat, consensus in etcd, leader-elected schedulers. A cloud platform that is itself a distributed system.

Example: a real Google Cloud architecture

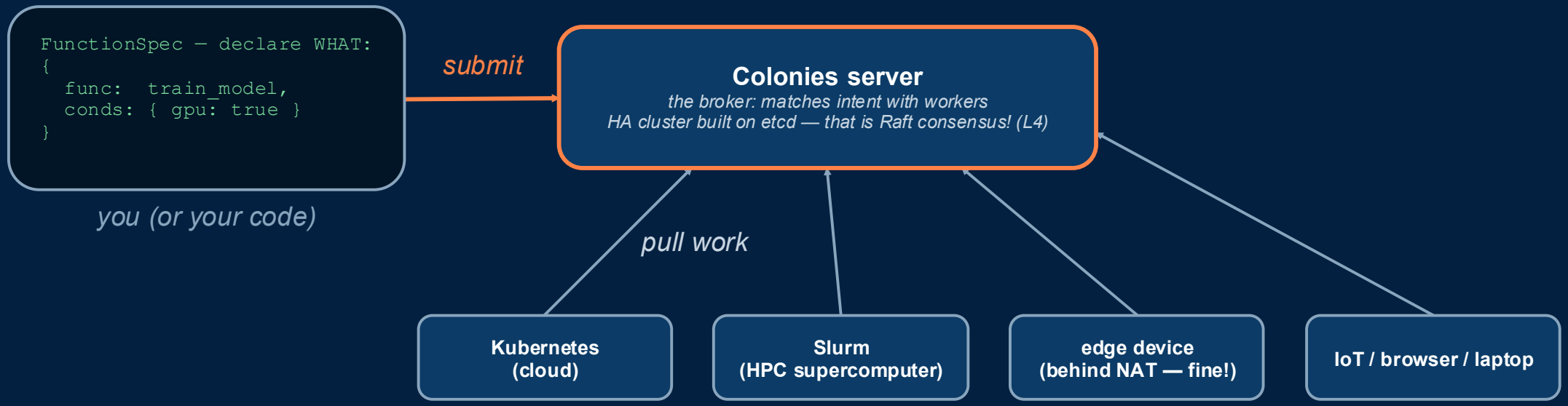


A perfectly ordinary Google Cloud web app — every box is a concept from this lecture, rentable by the hour. Knowing what each box costs and risks is exactly the skill this course trains.

*Beyond the cloud: E*dge, fog, and compute continuums

- **The cloud is far away.** A self-driving car at 110 km/h travels **3 metres** while waiting 100 ms for a cloud round-trip. Some decisions cannot commute.
- **Edge computing:** compute right next to the data — in the car, the base station, the factory robot. *(Your phone unlocking on your face = ML at the edge.)*
- **Fog computing:** the layer in between — gateways and local servers that pre-process before anything reaches the cloud. *(A wind farm filters terabytes of sensor data locally, ships only summaries north.)*
- **Edge–cloud compute continuum:** stop thinking "device or cloud" — one application flows across **device → edge → fog → cloud → HPC**, each piece running where it fits best.

Who keeps ten thousand machines busy? *Orchestration*



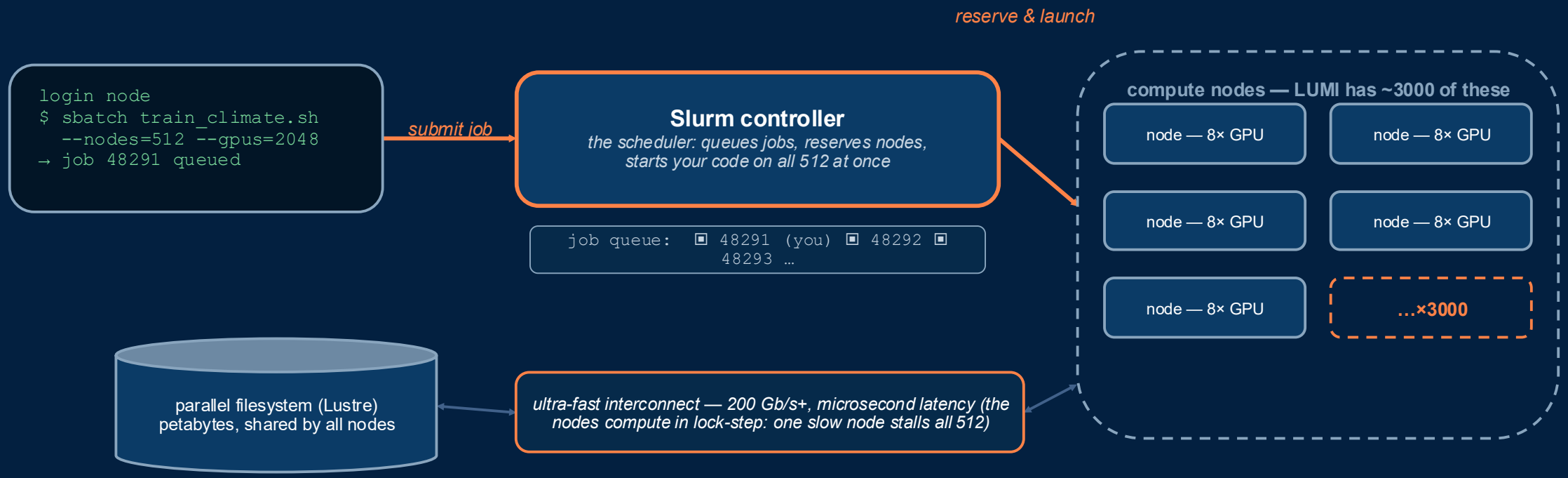
Executors — they know HOW to run things on their platform, and they PULL: no inbound ports, no firewall pain

ColonyOS — an open-source meta-orchestrator built by our research group here at LTU.
One spec, runs anywhere: cloud + HPC + edge + IoT = a compute continuum.

The heavyweights: HPC supercomputers

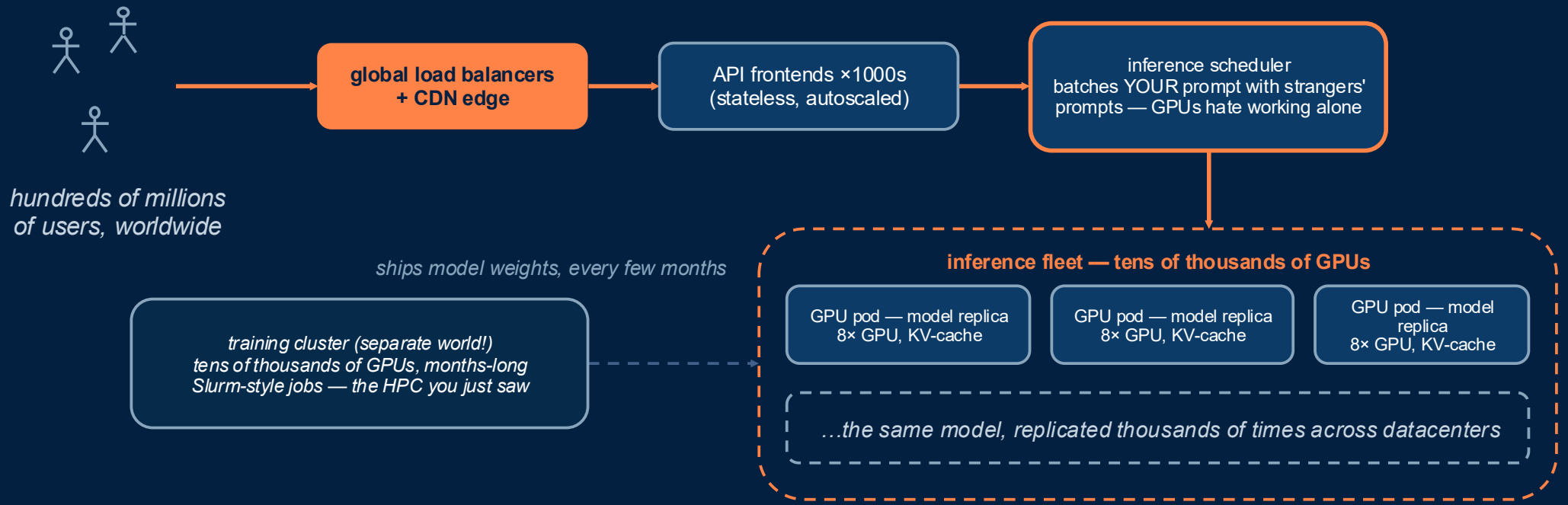
- **ChatGPT exists because of this:** GPT-class models are trained on **tens of thousands of GPUs for months** — one giant, tightly synchronised distributed computation.
- **Your weather forecast** (SMHI), climate models, drug discovery, crash simulations — all batch jobs queued on supercomputers like **LUMI** *(one of Europe's fastest, just across the border in Finland — Swedish students can apply for time on it!).*
- **Same problems, different trade-off:** HPC chases raw throughput with fast interconnects and tight clock sync — but one crashed node among 10,000 can still ruin a week-long job.
- Scheduled by **Slurm** — the HPC cousin of Kubernetes

Inside a supercomputer



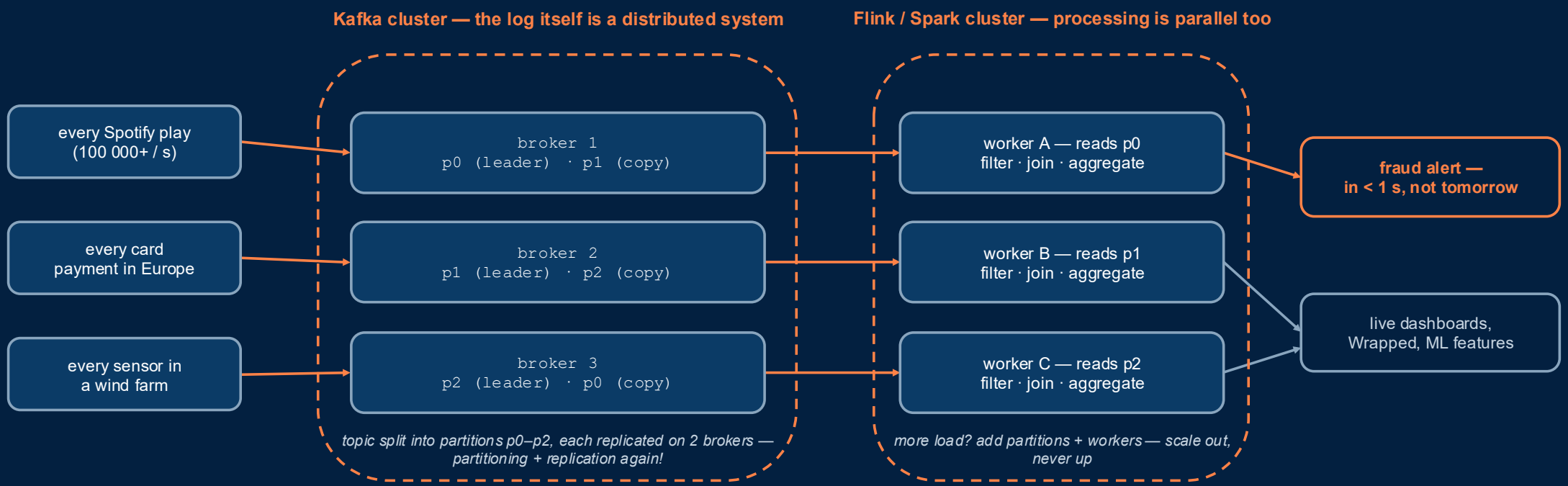
Same distributed-systems DNA, tuned for raw throughput: a scheduler instead of a load balancer, lock-step parallelism instead of loose coupling — and checkpointing instead of replication (compute is redone, not copied)

Case study: serving ChatGPT to a planet



Load balancing, replication, caching, batching, schedulers, failure detection — one chatbot, every single concept of this lecture. This is why AI engineers need this course.

Big data: Batch and Stream processing

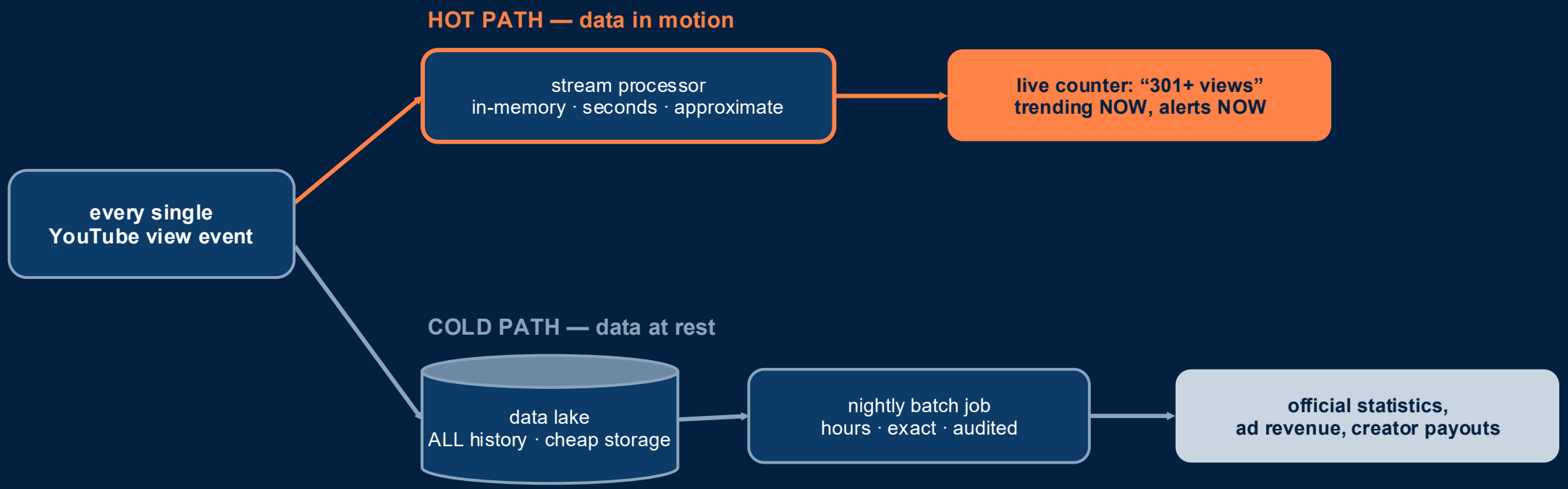


"Big data", two flavours: data at REST (store first, crunch nightly — batch) vs data in MOTION (process while it happens — stream)

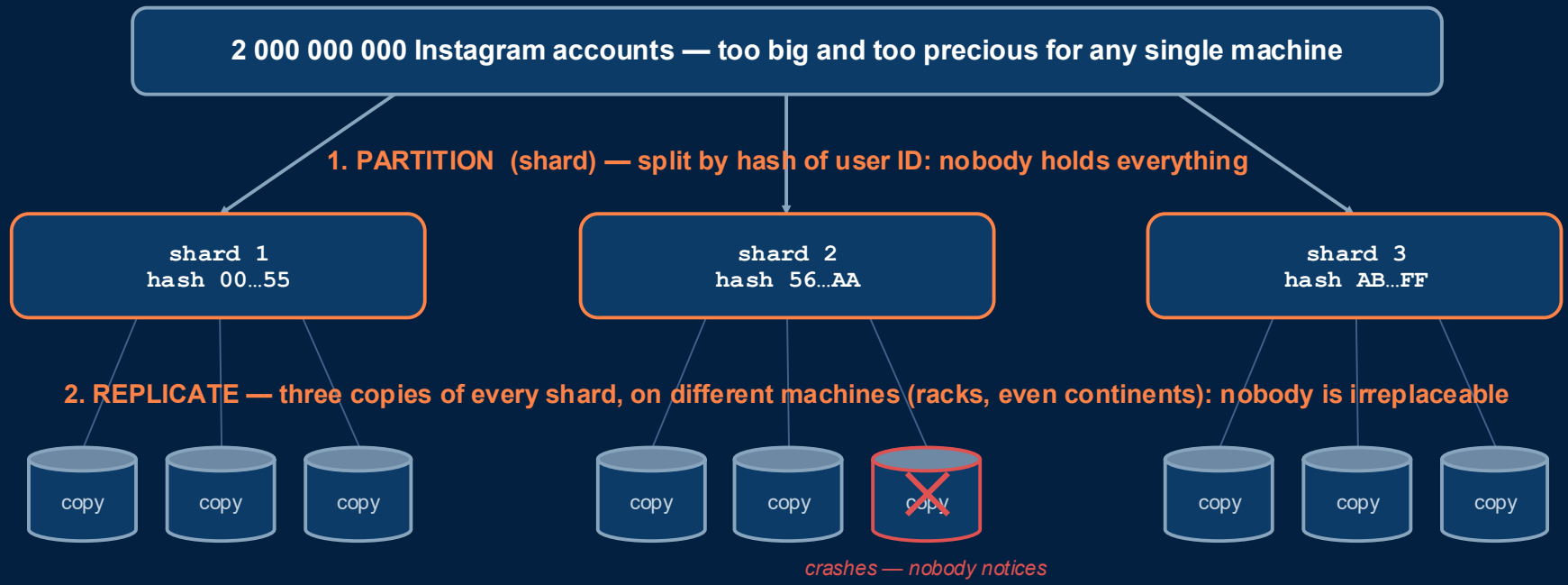
Event-driven architecture, industrial strength — partitioned logs, replicated brokers, parallel workers: every concept of this course, working one job → Lecture 6

Batch = the data is already sitting somewhere (a file, a table, a day's worth of logs). You define a *bounded* set, run a job over it, get an answer, job exits.
Stream = the data is arriving continuously and there is no end

Distributed analytics: Hot path and the cold path



Distributed databases

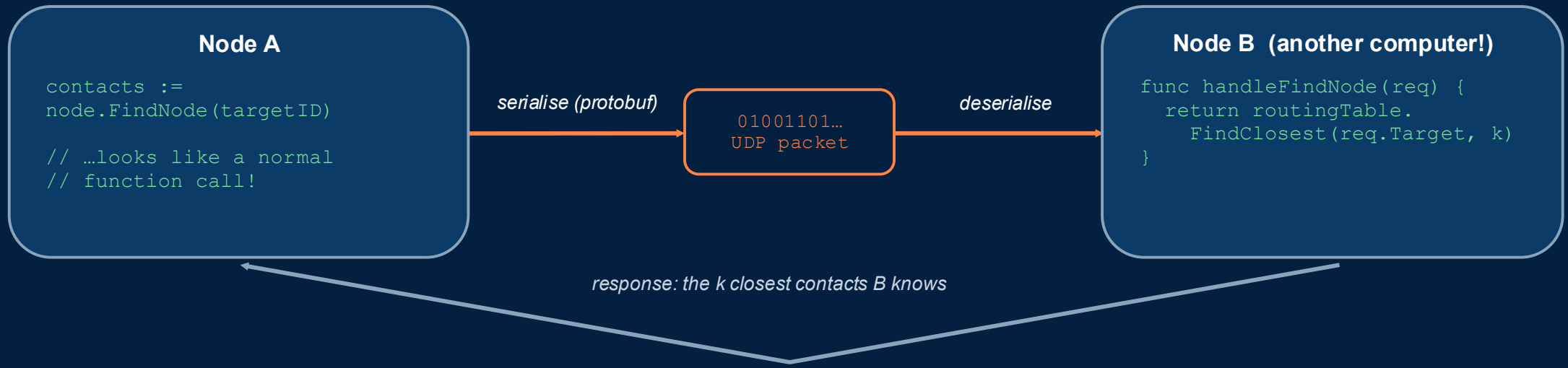


Cassandra, DynamoDB, Spanner, MongoDB — all built on these two moves. And so is your lab: the hash partitions the key space, and every object lives on k replicas. → Lectures 3 & 7

Part 4

The fundamental problems

Problem 1: Making machines talk — RPC



Your whole lab is exactly 4 of these RPCs:

`PING` `STORE` `FIND_NODE` `FIND_VALUE`

Sockets, serialisation, RPC, pub-sub

… and beneath RPC: the actual network

What you will touch in the lab

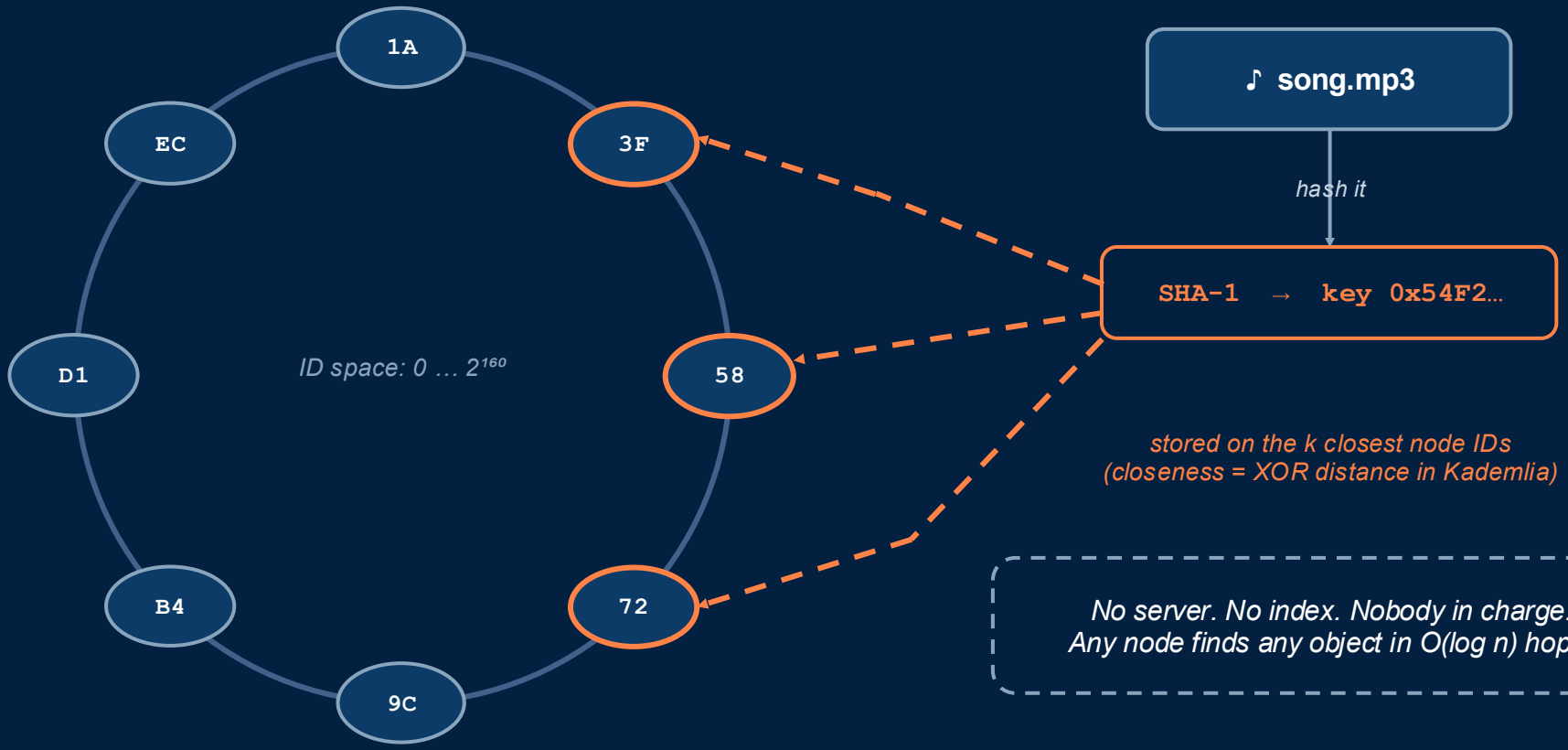
- Sockets — the bare-metal API of the internet
- **UDP** (lab!): fire-and-forget — fast, but packets drop, duplicate, reorder
- TCP: reliable & ordered — at the price of handshakes and head-of-line blocking
- NAT & firewalls: most peers can't be dialled — a defining constraint of P2P

How far is "far"?

same process	**~100 ns**
same datacenter	**~0.5 ms**
Luleå → Frankfurt	**~30 ms**
Luleå → Sydney	**~300 ms**

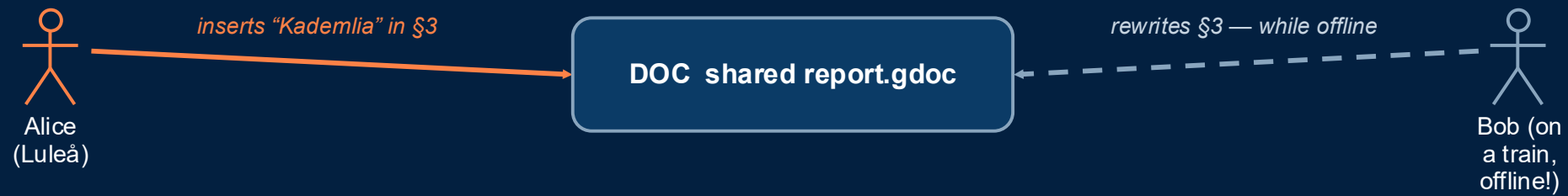
A remote call is **a million times slower** than a local one — every design in this course bends around that number.

Problem 2: Finding data with nobody in charge — DHTs



This runs BitTorrent (100M+ users) and IPFS — and it is exactly what YOU build in the lab

Problem 3: Agreeing on data — Google Docs



Same paragraph, same moment, no coordination possible.
Whose edit wins? In what order? Can we just lock the document? (please no)

When Bob comes online, BOTH converge to the SAME document — no locks, no lost edits.
Not magic: consistency models, CRDTs & Operational Transformation → Lectures 7–8

Consistency models: how “in sync” must copies be?

slower · must coordinate
“always correct”

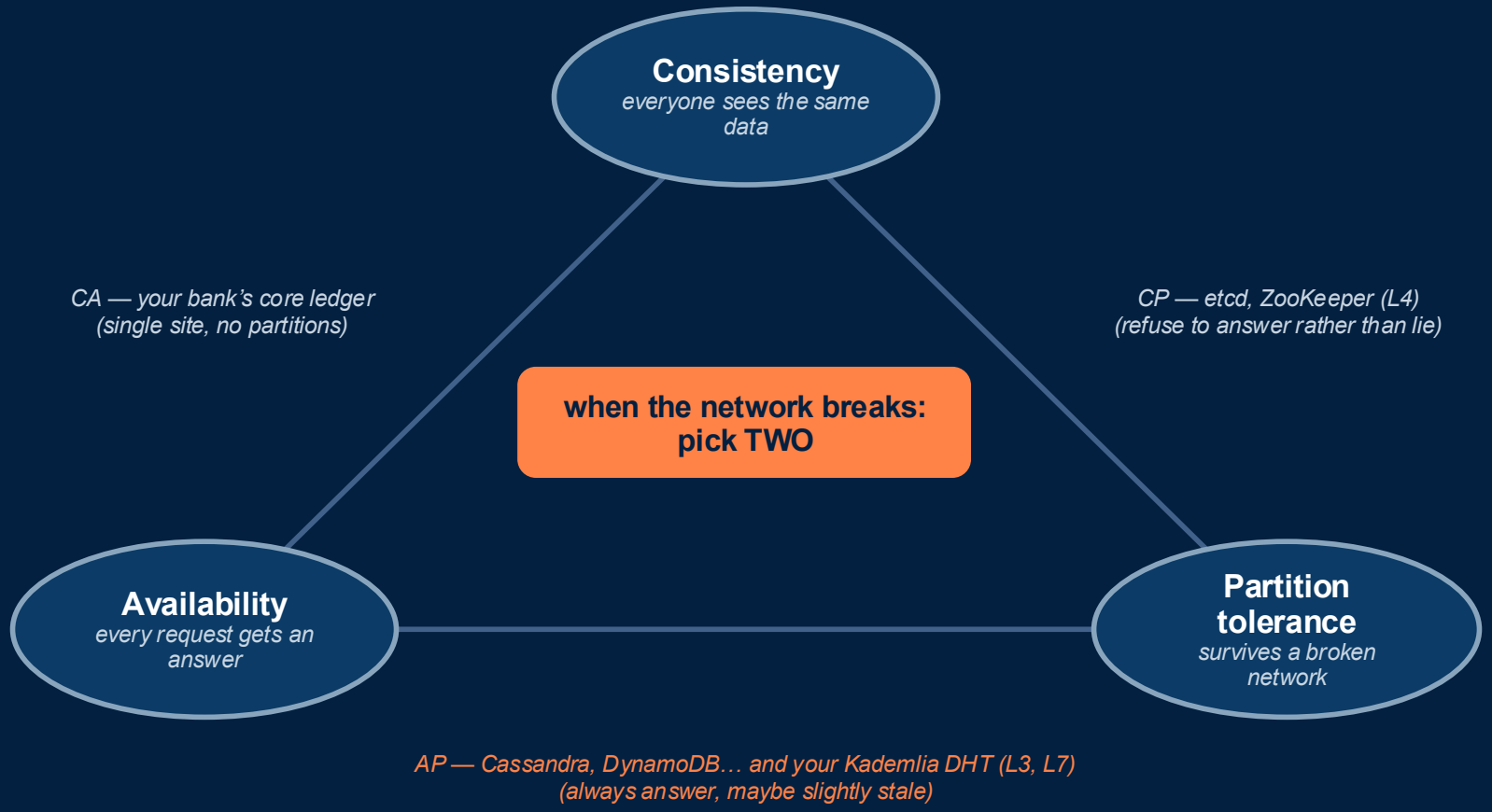
faster · always available
“correct soon”



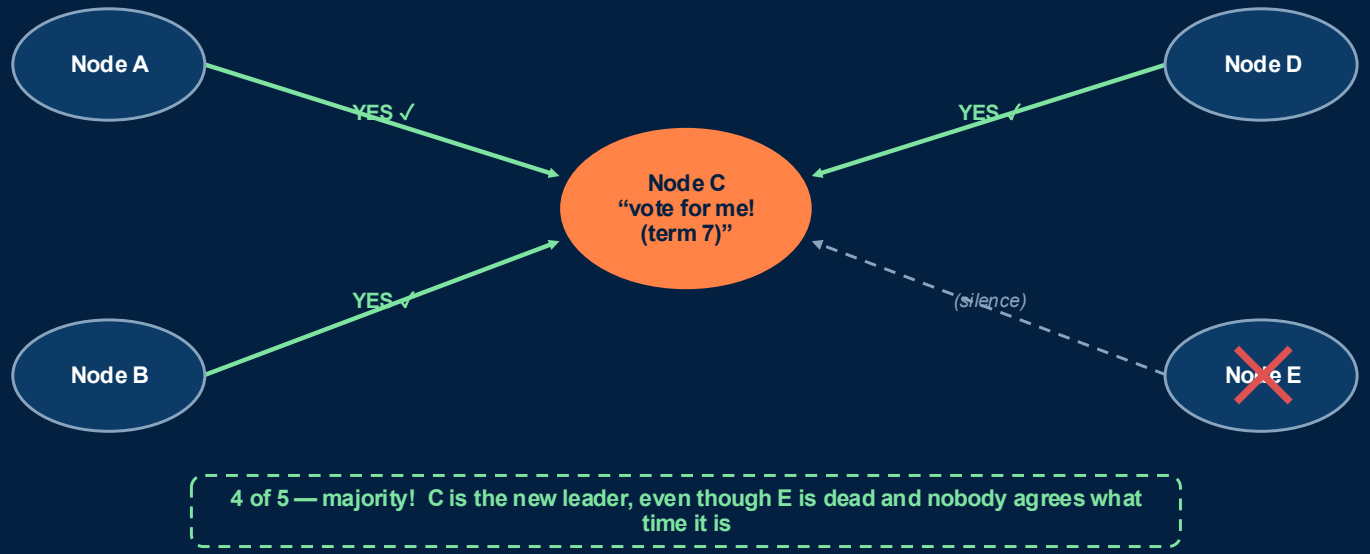
STRONG	CAUSAL	EVENTUAL
Your bank balance *a Swish payment must be visible everywhere, instantly — “roughly 500 kr” is not a balance*	**WhatsApp group chat** *the answer must never appear before the question — but exact global order? nobody notices*	**Instagram like counter** *1 847 or 1 851 likes? who cares — all replicas converge to the same number, eventually*

Pick the WEAKEST model your app survives — that is free speed and availability.
Your lab DHT is eventually consistent, and that is a feature → Lecture 7

CAP during a partition: consistency or availability



Problem 4: Consensus, picking a leader without a leader



This is how **etcd elects the leader of every Kubernetes cluster** — Raft, Paxos

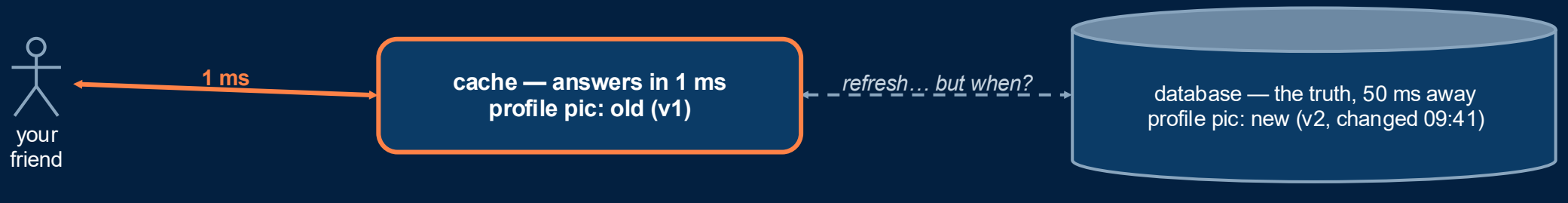
And if nodes can lie? Sounds impossible — yet **Bitcoin agrees among 15,000 strangers** *(Byzantine fault tolerance).*

Lab connection: your Kademlia network has no leader at all — a design choice; Lecture 4 shows what it buys you.

Problem 5: Time — “what happened first?”

- Two WhatsApp messages, sent “at the same time” from Luleå and Tokyo — which one is first? **Whose clock decides?**
- Every clock drifts. GPS time, NTP, atomic clocks — and still no two machines agree on now.
- The beautiful escape: stop asking what time it is and ask **what caused what** — logical clocks, vector clocks, happened-before.
- This single idea powers Git, CRDTs, distributed databases — and, yes, Google Docs.

Problem 6: Caching — fast, cheap, and slightly wrong



STALE! you changed your picture an hour ago
—
your friend still sees the old one

the eternal question: when to throw the copy away?
TTL (expire after a while) · write-through (update both) · event-based (tell every cache)

“There are only two hard things in CS: cache invalidation and naming things“ - Phil Karlton, Netscape

Problem 7: Failure modes — how exactly do things break?

✕ crash-stop
node dies, cleanly and forever — the polite failure

defence: timeouts + replication

↓ omission
messages silently lost — hello, UDP in your lab

defence: retries + acks + idempotence

… timing & partitions
alive but slow, or the network splits — is slow the same as dead?

defence: consensus, quorums (L4)

! Byzantine
the node LIES — sends different answers to different peers

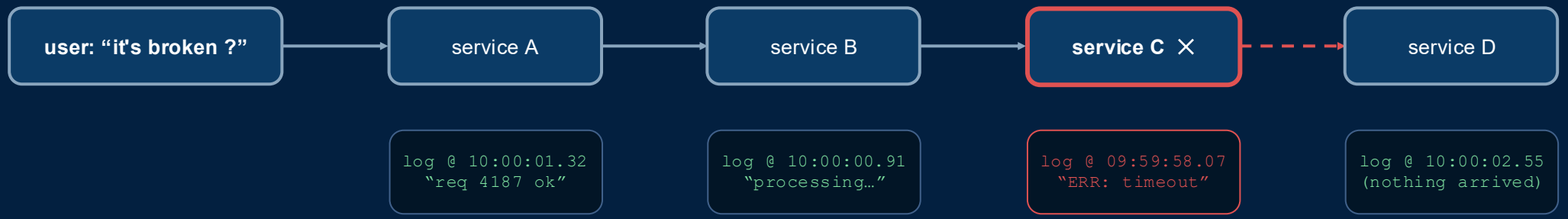
defence: BFT, PoW/PoS (L5)

harder to detect — and exponentially more expensive to defend against

The first design decision of ANY distributed system: which of these do you defend against?
Defending against more than you need = wasted complexity. Less than you need = headlines.

Your lab assumes crash-stop + omission (UDP drops packets — M4 makes you prove you survive it).
Bitcoin must survive Byzantine. That single assumption changes the whole design.

Problem 8: Observability — failures spread over 50 machines



Four machines, four log files, four clocks that disagree — C’s “09:59:58” happened AFTER B’s “10:00:00”!
There is no debugger to attach, no single screen to look at, and print( ) is spread over the planet.

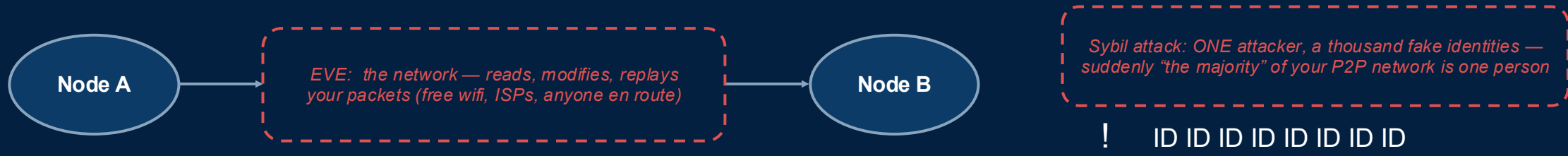
correlation IDs + distributed tracing

metrics + alerting (Prometheus)

logical clocks — order without time (L7)

In the lab this hits you in week 2: a bug across 50 containers, reproducible only sometimes.
Your weapons: unit tests with emulated networks (M4), docker compose logs, and Go’s race detector.

Problem 9: Security & trust — the network is hostile



In a P2P system every peer is a stranger. Who do you trust when there is no one to ask?

encrypt: TLS / Noise (L6)

sign: Ed25519 (ColonyOS!)

name by hash: tamper-evident

majority of strangers: BFT (L5)

Your lab has elegant built-in integrity: objects are named by their hash — if a stranger's node tampers with your data, the hash no longer matches. You get tamper-evidence for free. (Bitcoin scales that idea to money.)

Problem 10: Overload



requests time out → every client RETRIES → load doubles → more timeouts → …
the thundering herd: your users’ impatience becomes a self-inflicted DDoS

- **rate limit — “HTTP 429, slow down please”**
- **backpressure — tell the producer to back off**
- **circuit breaker — stop calling a dying service**
- **load shedding — drop some, save the rest**
- **backoff + jitter — retry politely**

Overload control is genuinely hard. In your lab: 1000 emulated nodes retrying lookups against dropped UDP packets — choose your retry strategy with care, or herd yourself.

Part 5

The course

How you will learn (7.5 ECTS)

Lectures

- Theory, design principles, real systems
- On campus + Zoom
- Practice quizzes *(ungraded)*

Lab — 3 ECTS

- Build a P2P distributed data store
- Groups of 3, agile sprints
- The deep, hands-on core of the course

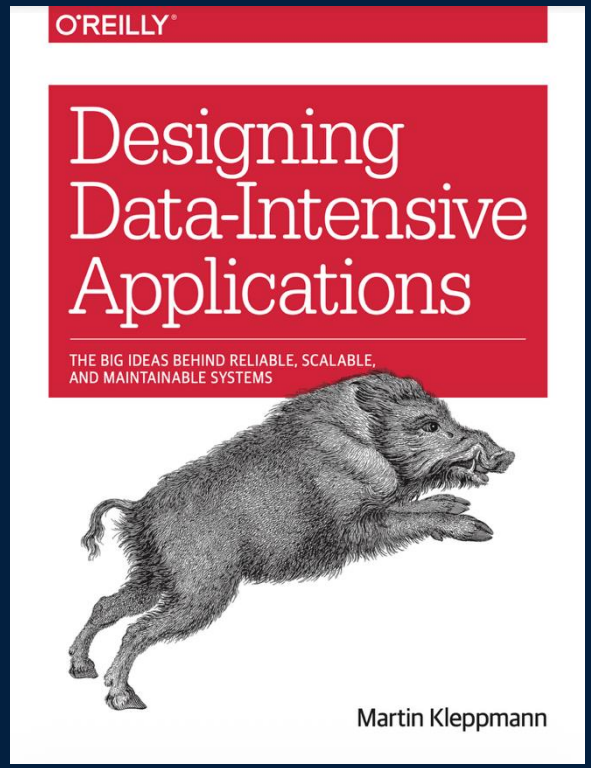
Seminar — 1.5 ECTS

- Present one scientific paper (25 min)
- Groups of 3, attendance required

Home exam — 3 ECTS

- Trade-off judgement, not memorising: predict, measure, plot

Martin Kleppmann Distributed Systems lecture series
https://www.youtube.com/playlist?list=PLeKd45zvjcDFUEv_ohr_HdUFe97RItdiB



The lectures — when, where, who

Ten lectures, four weeks · on campus and live on Zoom · Note NO recordings

L1 — Course introduction & characteristics of DS
Tue 1 Sep · 14:45–16:15 · A110 · **today**
Johan Christoffer

L2 — Developing distributed algorithms in Golang
Fri 4 Sep · 10:15–11:45 · A110
Johan

L3 — Communication and protocols
Mon 7 Sep · 14:45–16:15 · A110
Carl

L4 — Distributed hash tables (DHT)
Wed 9 Sep · 13:00–14:30 · A211
Online Only
Christoffer

L5 — Leader election and distributed consensus
Fri 11 Sep · 10:15–11:45 · A211
Online Only
Christoffer

L6 — Blockchains and Byzantine fault tolerance
Mon 14 Sep · 13:00–14:30 · A211
Online Only
Christoffer

L7 — Data synchronisation and replication
Wed 16 Sep · 10:15–11:45 · A211
Johan

L8 — Orchestration
Fri 18 Sep · 10:15–11:45 · A211
Johan

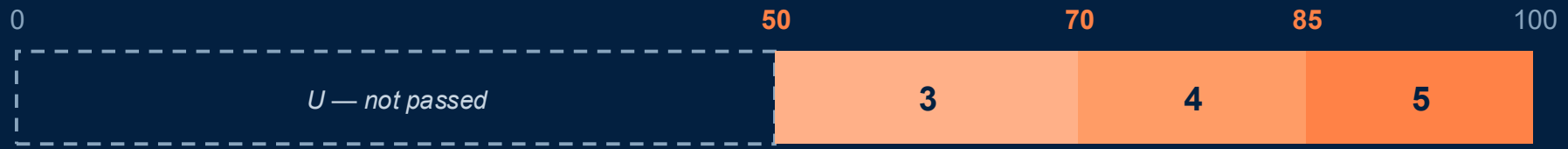
L9 — Distributed gaming backends
Tue 22 Sep · 14:45–16:15 · A211
Carl

L10 — Reserve
Fri 25 Sep · 10:15–11:45 · A211
Johan

Grading

Your final score =





Seminars

- In groups of 3: pick a **peer-reviewed** paper (IEEE Xplore, ACM DL, Springer — free via the LTU library) and present it: **25 minutes** (20 talk + 5 discussion).
- Face-to-face, broadcast on Zoom; attendance at the seminar sessions is required.
- Why? The systems in this course were all papers once — Kademlia (2002), Raft (2014), Bitcoin (2008), Spanner (2012).

Questions?

johan.kristiansson@ltu.se · carl.borngrund@ltu.se · christoffer.fink@ltu.se


LULEÅ
TEKNISKA
UNIVERSITET